

Training Neural Networks in PyTorch: A Comprehensive Guide

This notebook provides an in-depth guide to training neural networks effectively using PyTorch. We cover everything from the fundamental training loop to advanced techniques for optimization, regularization, and monitoring.

```
In [ ]: import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, Dataset, random_split, TensorDataset
from torch.optim.lr_scheduler import StepLR, MultiStepLR, ExponentialLR, CosineAnnealingLR, ReduceLROnPlateau
from torch.nn.utils import clip_grad_norm_, clip_grad_value_
from torch.utils.tensorboard import SummaryWriter
import matplotlib.pyplot as plt
import numpy as np
import os
import time
import math

# Set random seed for reproducibility
torch.manual_seed(42)
np.random.seed(42)

# Device configuration
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Create output directory for plots, models, and logs
output_dir = "04_training_neural_networks_outputs"
os.makedirs(output_dir, exist_ok=True)
os.makedirs(os.path.join(output_dir, "runs"), exist_ok=True)
os.makedirs(os.path.join(output_dir, "saved_models"), exist_ok=True)
```

1. Introduction to Neural Network Training

Neural network training is the process of teaching a model to learn patterns from data through an iterative optimization process. The core components include:

- **Model:** The neural network architecture (e.g., MLP, CNN) defined using `nn.Module`
- **Data:** Input features and target labels, typically split into training, validation, and test sets
- **Loss Function:** Measures the discrepancy between predictions and true values
- **Optimizer:** Algorithm that adjusts model parameters to minimize the loss function

The training process involves:

- **Epochs:** Complete passes through the entire training dataset
- **Batches:** The dataset is divided into smaller subsets for more manageable processing

2. Preparing Your Data with Dataset and DataLoader

PyTorch provides convenient utilities for handling data efficiently:

```
In [ ]: class MyCustomDataset(Dataset):
        """Example of a custom Dataset."""
        def __init__(self, num_samples=1000, input_features=10, num_classes=2, transform=None):
            # Generate some random data for demonstration
            self.data = torch.randn(num_samples, input_features)
            self.targets = torch.randint(0, num_classes, (num_samples,))
            self.transform = transform
            print(f"CustomDataset: Created {num_samples} samples.")

        def __len__(self):
            return len(self.data)

        def __getitem__(self, idx):
            sample = self.data[idx]
            target = self.targets[idx]
            if self.transform:
                sample = self.transform(sample)
            return sample, torch.tensor(target, dtype=torch.long)
```

```
# Using the custom dataset
print("--- Custom Dataset Example ---")
custom_train_dataset = MyCustomDataset(num_samples=100, input_features=5, num_classes=3)
sample_data, sample_target = custom_train_dataset[0]
print(f"First sample data shape: {sample_data.shape}, target: {sample_target}")

# Using DataLoader with the custom dataset
custom_train_loader = DataLoader(custom_train_dataset, batch_size=32, shuffle=True)
print(f"Number of batches in custom_train_loader: {len(custom_train_loader)}")
for i, (batch_data, batch_targets) in enumerate(custom_train_loader):
    print(f"Batch {i+1} data shape: {batch_data.shape}, targets shape: {batch_targets.shape}")
    if i == 0: # Print only first batch details
        break
```

```
In [ ]: # Using torchvision for a standard dataset (MNIST)
print("\n--- torchvision MNIST Dataset and DataLoader Example ---")
mnist_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,)) # MNIST mean and std
])
mnist_train_dataset = torchvision.datasets.MNIST(
    root='./data', train=True, download=True, transform=mnist_transform
)
mnist_train_loader = DataLoader(mnist_train_dataset, batch_size=64, shuffle=True)
print(f"Number of samples in MNIST training set: {len(mnist_train_dataset)}")
print(f"Number of batches in MNIST train_loader: {len(mnist_train_loader)}")
mnist_batch_data, mnist_batch_targets = next(iter(mnist_train_loader))
print(f"MNIST first batch data shape: {mnist_batch_data.shape}, targets shape: {mnist_batch_targets.shape}")
print("`Dataset` manages data samples, `DataLoader` provides batches for training.")
```

3. Defining a Neural Network

Let's define a simple neural network that we'll use throughout this tutorial:

```
In [ ]: class SimpleNN(nn.Module):
    def __init__(self, input_size=28*28, hidden_size=128, num_classes=10, use_dropout=False, use_bn=False):
        super(SimpleNN, self).__init__()
```

```

self.use_dropout = use_dropout
self.use_bn = use_bn

self.fc1 = nn.Linear(input_size, hidden_size)
if self.use_bn:
    self.bn1 = nn.BatchNorm1d(hidden_size)
self.relu = nn.ReLU()
if self.use_dropout:
    self.dropout = nn.Dropout(0.5)
self.fc2 = nn.Linear(hidden_size, num_classes)

def forward(self, x):
    x = x.view(x.size(0), -1) # Flatten the input
    x = self.fc1(x)
    if self.use_bn:
        x = self.bn1(x)
    x = self.relu(x)
    if self.use_dropout:
        x = self.dropout(x)
    x = self.fc2(x)
    return x

# Create a model instance
model = SimpleNN().to(device)
print("Neural Network Architecture:")
print(model)

```

4. The Essential Training Loop

The training loop is the core of neural network training. Here's how one iteration works:

```

In [ ]: # Dummy data for demonstration
dummy_inputs = torch.randn(64, 28*28).to(device) # Batch of 64 flattened images
dummy_targets = torch.randint(0, 10, (64,)).to(device) # 64 labels for 10 classes

model = SimpleNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

```

```
print("Running one iteration of the training loop...")

# 1. Set model to training mode
model.train()

# 2. Zero gradients (typically at start of batch loop)
optimizer.zero_grad()

# 3. Forward pass: Getting predictions
outputs = model(dummy_inputs)
print(f" Output shape: {outputs.shape}")

# 4. Calculate the loss
loss = criterion(outputs, dummy_targets)
print(f" Calculated loss: {loss.item():.4f}")

# 5. Backward pass: Computing gradients
loss.backward()
print(f" Gradients computed (e.g., model.fc1.weight.grad is not None: {model.fc1.weight.grad is not None})")

# 6. Optimizer step: Updating weights
optimizer.step()
print(f" Optimizer step taken (weights updated).")
print("This forms one iteration. A full epoch repeats this for all batches.")
```

5. Validation: Evaluating Model Performance

Validation helps monitor overfitting and assess how well the model generalizes to unseen data.

```
In [ ]: def get_mnist_loaders(batch_size=64, validation_split=0.1):
        """Helper function to get MNIST data loaders"""
        transform = transforms.Compose([
            transforms.ToTensor(),
            transforms.Normalize((0.1307,), (0.3081,))
        ])
        full_train_dataset = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform)
        test_dataset = torchvision.datasets.MNIST(root='./data', train=False, download=True, transform=transform)

        num_train = len(full_train_dataset)
```

```

val_size = int(validation_split * num_train)
train_size = num_train - val_size

train_subset, val_subset = random_split(full_train_dataset, [train_size, val_size])

train_loader = DataLoader(train_subset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_subset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
return train_loader, val_loader, test_loader

# Get data loaders
train_loader, val_loader, test_loader = get_mnist_loaders()
print(f"Train batches: {len(train_loader)}, Val batches: {len(val_loader)}, Test batches: {len(test_loader)}")

```

```

In [ ]: # Validation loop demonstration
model = SimpleNN().to(device)
criterion = nn.CrossEntropyLoss()

print("Running one validation epoch...")
# 1. Set model to evaluation mode
model.eval()
val_loss = 0.0
correct_predictions = 0
total_samples = 0

# 2. Disable gradient computation
with torch.no_grad():
    for inputs, targets in val_loader:
        inputs, targets = inputs.to(device), targets.to(device)

        outputs = model(inputs)
        loss = criterion(outputs, targets)

        val_loss += loss.item() * inputs.size(0)
        _, predicted_classes = outputs.max(1)
        total_samples += targets.size(0)
        correct_predictions += predicted_classes.eq(targets).sum().item()

    # Print only first batch details
    if total_samples == inputs.size(0):

```

```

        print(f" Validation batch: Loss={loss.item():.4f}")

        # Break after a few batches for demo
        if total_samples >= 256:
            break

epoch_loss = val_loss / total_samples
epoch_accuracy = correct_predictions / total_samples
print(f"Validation Results: Average Loss: {epoch_loss:.4f}, Accuracy: {epoch_accuracy*100:.2f}%")
print("`model.eval()` and `torch.no_grad()` are crucial for correct validation.")

```

6. Saving and Loading Models

It's essential to save your trained model for later use or to resume training.

```

In [ ]: # Create and train a model briefly for saving
model_to_save = SimpleNN().to(device)
optimizer = optim.Adam(model_to_save.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()

# Simulate some training
dummy_input = torch.randn(10, 28*28).to(device)
dummy_target = torch.randint(0, 10, (10,)).to(device)
for _ in range(2): # Few dummy steps
    optimizer.zero_grad()
    loss = criterion(model_to_save(dummy_input), dummy_target)
    loss.backward()
    optimizer.step()

model_path = os.path.join(output_dir, "saved_models", "simple_nn_statedict.pth")
checkpoint_path = os.path.join(output_dir, "saved_models", "checkpoint.pth")

# --- Saving and Loading State Dictionary (Recommended) ---
print("--- Saving and Loading Model State Dictionary ---")
torch.save(model_to_save.state_dict(), model_path)
print(f"Model state_dict saved to: {model_path}")

# Load the state_dict
model_loaded_state_dict = SimpleNN().to(device) # Create a new instance

```

```
model_loaded_state_dict.load_state_dict(torch.load(model_path))
model_loaded_state_dict.eval() # Set to evaluation mode
print("Model loaded from state_dict successfully.")
```

```
In [ ]: # --- Saving and Loading Checkpoints (for resuming training) ---
print("\n--- Saving and Loading Checkpoints ---")
epoch = 5
current_loss = 0.123
checkpoint = {
    'epoch': epoch + 1, # Save next epoch to start from
    'model_state_dict': model_to_save.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'loss': current_loss,
}
torch.save(checkpoint, checkpoint_path)
print(f"Checkpoint saved to: {checkpoint_path}")

# Load from checkpoint
model_for_resume = SimpleNN().to(device)
optimizer_for_resume = optim.Adam(model_for_resume.parameters(), lr=0.001)

loaded_checkpoint = torch.load(checkpoint_path)
model_for_resume.load_state_dict(loaded_checkpoint['model_state_dict'])
optimizer_for_resume.load_state_dict(loaded_checkpoint['optimizer_state_dict'])
start_epoch = loaded_checkpoint['epoch']
previous_loss = loaded_checkpoint['loss']
model_for_resume.train() # Set to train mode to resume training
print(f"Checkpoint loaded. Resuming from epoch {start_epoch}, previous loss: {previous_loss:.4f}")
print("Always save `state_dict` for portability and flexibility.")
```

7. Learning Rate Scheduling

Adjusting learning rate during training can improve convergence and performance.

```
In [ ]: # Demonstrate different learning rate schedulers
model = SimpleNN(hidden_size=32).to(device) # Smaller model for quick demo
optimizer = optim.SGD(model.parameters(), lr=0.1)
num_epochs_lr_demo = 15
```



```
schedulers_to_test = {
    "StepLR (step=5, gamma=0.5)": StepLR(optimizer, step_size=5, gamma=0.5),
    "ExponentialLR (gamma=0.85)": ExponentialLR(optimizer, gamma=0.85),
    "CosineAnnealingLR (T_max=15)": CosineAnnealingLR(optimizer, T_max=num_epochs_lr_demo),
}

plt.figure(figsize=(12, 6))
for name, scheduler in schedulers_to_test.items():
    # Reset optimizer for each scheduler test
    optimizer = optim.SGD(model.parameters(), lr=0.1)

    # Re-create scheduler with new optimizer
    if "StepLR" in name:
        current_scheduler = StepLR(optimizer, step_size=5, gamma=0.5)
    elif "ExponentialLR" in name:
        current_scheduler = ExponentialLR(optimizer, gamma=0.85)
    elif "CosineAnnealingLR" in name:
        current_scheduler = CosineAnnealingLR(optimizer, T_max=num_epochs_lr_demo)

    lr_history = []
    for epoch in range(num_epochs_lr_demo):
        lr_history.append(optimizer.param_groups[0]['lr'])
        current_scheduler.step()

    plt.plot(lr_history, label=name)

plt.title("Learning Rate Schedules")
plt.xlabel("Epoch")
plt.ylabel("Learning Rate")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

print("Different schedulers provide various learning rate decay patterns.")
```

8. Regularization Techniques

Regularization helps prevent overfitting by constraining the model's capacity.

```
In [ ]: # --- L2 Regularization (Weight Decay) ---
print("--- L2 Regularization (Weight Decay) ---")
model_wd = SimpleNN().to(device)
# Add weight_decay to the optimizer
optimizer_wd = optim.Adam(model_wd.parameters(), lr=0.001, weight_decay=1e-4)
print(f"Optimizer with weight_decay (L2 penalty): weight_decay={optimizer_wd.param_groups[0]['weight_decay']}")

# --- Dropout ---
print("\n--- Dropout ---")
model_dropout = SimpleNN(use_dropout=True).to(device)
print("Model with Dropout layer:")
print(model_dropout)

# Demonstrate dropout behavior
model_dropout.train()
dummy_input_reg = torch.randn(5, 28*28).to(device)
output_train = model_dropout(dummy_input_reg)
print(f"Output with dropout (train mode): {output_train[0,:5]}")

model_dropout.eval()
output_eval = model_dropout(dummy_input_reg)
print(f"Output with dropout (eval mode): {output_eval[0,:5]}")
print("During training, dropout randomly zeros elements. During evaluation, it's disabled.")
```

9. Gradient Clipping

Prevents exploding gradients by capping their norm or value.

```
In [ ]: model_gc = SimpleNN().to(device)
optimizer_gc = optim.SGD(model_gc.parameters(), lr=0.01)
criterion_gc = nn.MSELoss()
dummy_input_gc = torch.randn(5, 28*28).to(device)
dummy_target_gc = torch.randn(5, 10).to(device)

optimizer_gc.zero_grad()
outputs_gc = model_gc(dummy_input_gc)
```

```

loss_gc = criterion_gc(outputs_gc, dummy_target_gc)
loss_gc.backward()

# Print norm of gradients before clipping
original_grad_norm = model_gc.fc1.weight.grad.norm().item()
print(f"Original grad norm for fc1.weight: {original_grad_norm:.4f}")

# Clip gradient norm
max_norm = 1.0
total_norm_clipped = clip_grad_norm_(model_gc.parameters(), max_norm=max_norm)
print(f"Total norm of gradients after clipping by norm to {max_norm}: {total_norm_clipped:.4f}")
clipped_grad_norm = model_gc.fc1.weight.grad.norm().item()
print(f"Clipped grad norm for fc1.weight: {clipped_grad_norm:.4f}")

# Demonstrate gradient value clipping
optimizer_gc.zero_grad()
outputs_gc = model_gc(dummy_input_gc)
loss_gc = criterion_gc(outputs_gc, dummy_target_gc)
loss_gc.backward()
clip_val = 0.1
clip_grad_value_(model_gc.parameters(), clip_value=clip_val)
print(f"\nAfter clipping values to +/- {clip_val}:")
print(f"  Min grad value: {model_gc.fc1.weight.grad.min().item():.4f}")
print(f"  Max grad value: {model_gc.fc1.weight.grad.max().item():.4f}")
print("Gradient clipping is applied after .backward() and before .step().")

```

10. Batch Normalization

Normalizes activations, stabilizes and accelerates training.

```

In [ ]: model_with_bn = SimpleNN(use_bn=True).to(device)
print("Model with Batch Normalization layer:")
print(model_with_bn)

dummy_input_bn = torch.randn(5, 28*28).to(device)

# Behavior in train() mode
model_with_bn.train()
output_bn_train = model_with_bn(dummy_input_bn)

```

```

print(f"\nOutput with BatchNorm (train mode) shape: {output_bn_train.shape}")
print(f"Running mean of bn1 after one forward pass (train): {model_with_bn.bn1.running_mean[0].item():.4f}")

# Behavior in eval() mode
model_with_bn.eval()
output_bn_eval = model_with_bn(dummy_input_bn)
print(f"Output with BatchNorm (eval mode) shape: {output_bn_eval.shape}")
print("`model.train()` and `model.eval()` are crucial for BatchNorm to work correctly.")
print("In train mode, BN uses batch statistics and updates running mean/var.")
print("In eval mode, BN uses computed running mean/var and does not update them.")

```

11. Complete Training Pipeline Example

Let's put everything together in a complete training pipeline that demonstrates the concepts we've learned.

```

In [ ]: # Configuration
LEARNING_RATE = 0.001
BATCH_SIZE = 128
NUM_EPOCHS = 2 # Low for demo, typically 10-100+
HIDDEN_SIZE = 256
WEIGHT_DECAY = 1e-5
CLIP_GRAD_NORM = 1.0

# Data Loading
train_loader, val_loader, test_loader = get_mnist_loaders(batch_size=BATCH_SIZE)
print(f"Train batches: {len(train_loader)}, Val batches: {len(val_loader)}")

# Model Definition with regularization
model = SimpleNN(28*28, HIDDEN_SIZE, 10, use_dropout=True, use_bn=True).to(device)

# Initialize weights
for m in model.modules():
    if isinstance(m, nn.Linear):
        nn.init.kaiming_normal_(m.weight, nonlinearity='relu')
        if m.bias is not None:
            nn.init.zeros_(m.bias)

# Loss, Optimizer, Scheduler
criterion = nn.CrossEntropyLoss()

```

```
optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE, weight_decay=WEIGHT_DECAY)
scheduler = ReduceLROnPlateau(optimizer, 'min', patience=3, factor=0.5, verbose=True)

print("Model and training setup complete.")
```

```
In [ ]: # Training Loop
history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': [], 'lr': []}

for epoch in range(NUM_EPOCHS):
    print(f"\nEpoch {epoch+1}/{NUM_EPOCHS}")

    # Training Phase
    model.train()
    train_loss = 0.0
    train_correct = 0
    train_total = 0

    for batch_idx, (inputs, targets) in enumerate(train_loader):
        inputs, targets = inputs.to(device), targets.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()

        # Gradient Clipping
        if CLIP_GRAD_NORM > 0:
            clip_grad_norm_(model.parameters(), max_norm=CLIP_GRAD_NORM)

        optimizer.step()

        train_loss += loss.item() * inputs.size(0)
        _, predicted = outputs.max(1)
        train_total += targets.size(0)
        train_correct += predicted.eq(targets).sum().item()

        if (batch_idx + 1) % 100 == 0:
            batch_acc = predicted.eq(targets).sum().item() / targets.size(0)
            print(f' Batch {batch_idx+1}/{len(train_loader)} | Loss: {loss.item():.4f} | Acc: {batch_acc*100:.2f}%')
```

```
avg_train_loss = train_loss / train_total
avg_train_acc = train_correct / train_total

# Validation Phase
model.eval()
val_loss = 0.0
val_correct = 0
val_total = 0

with torch.no_grad():
    for inputs, targets in val_loader:
        inputs, targets = inputs.to(device), targets.to(device)
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        val_loss += loss.item() * inputs.size(0)
        _, predicted = outputs.max(1)
        val_total += targets.size(0)
        val_correct += predicted.eq(targets).sum().item()

avg_val_loss = val_loss / val_total
avg_val_acc = val_correct / val_total

# Update scheduler
current_lr = optimizer.param_groups[0]['lr']
scheduler.step(avg_val_loss)

# Store history
history['train_loss'].append(avg_train_loss)
history['train_acc'].append(avg_train_acc)
history['val_loss'].append(avg_val_loss)
history['val_acc'].append(avg_val_acc)
history['lr'].append(current_lr)

print(f" Train Loss: {avg_train_loss:.4f}, Train Acc: {avg_train_acc*100:.2f}%")
print(f" Val Loss: {avg_val_loss:.4f}, Val Acc: {avg_val_acc*100:.2f}%")
print(f" Learning Rate: {current_lr:.6f}")

print("\nTraining completed!")
```

```
In [ ]: # Plot training history
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.plot(history['train_loss'], label='Train Loss')
plt.plot(history['val_loss'], label='Validation Loss')
plt.title('Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.subplot(1, 3, 2)
plt.plot(history['train_acc'], label='Train Accuracy')
plt.plot(history['val_acc'], label='Validation Accuracy')
plt.title('Accuracy over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

plt.subplot(1, 3, 3)
plt.plot(history['lr'], label='Learning Rate')
plt.title('Learning Rate over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Learning Rate')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

print("Training history visualization complete.")
```

Conclusion

This notebook covered the essential aspects of training neural networks in PyTorch:

1. **Data Preparation:** Using Dataset and DataLoader for efficient data handling
2. **Training Loop:** The core iterative process of training
3. **Validation:** Evaluating model performance during training
4. **Model Persistence:** Saving and loading models and checkpoints
5. **Learning Rate Scheduling:** Dynamically adjusting learning rates
6. **Regularization:** Techniques to prevent overfitting (dropout, weight decay)
7. **Gradient Clipping:** Preventing exploding gradients
8. **Batch Normalization:** Stabilizing and accelerating training
9. **Complete Pipeline:** Integrating all concepts in a real training scenario

These techniques form the foundation for training effective neural networks. In practice, you'll combine multiple techniques based on your specific problem, dataset size, and computational resources.

Next Steps:

- Experiment with different architectures (CNNs, RNNs, Transformers)
- Try different optimizers (SGD, Adam, RMSprop)
- Explore advanced regularization techniques
- Learn about transfer learning and fine-tuning pre-trained models